



Context Course documentation

Quiz 2: Multi-Agent Workflows ▾



Quiz 2: Multi-Agent Workflows

[Copy page](#)

Test your ability to design and implement multi-agent workflows.

Question 1: When is the overhead of subagents worth it?

- ☐ Always spawn as many subagents as possible for maximum parallelism
- ☒ Use subagents when tasks are independent and parallelizable, and the overhead is justified by the task complexity

Correct! Correct! Subagent workflows consume more tokens than single-agent runs. Only use them when the parallelism benefit outweighs the overhead.

- ☐ Subagents are only for research tasks; never use for implementation
- ☐ Subagents should always share the same context window as the parent

[Submit](#)**You got all the answers!**

Question 2: How do parent agents share context with subagents?

- ☐ Subagents can directly access the parent agent's memory and variables
- ☒ Pass small context in the task description; for large data, write to a file and reference the path

Correct! Correct! Subagents can't access parent memory. Use task descriptions for small context and files for larger data.

- ☐ Use global variables that all subagents can read
- ☐ There's no way to share data between parent and subagents

[Ask HuggingChat](#)

You got all the answers!

Question 3: How should you design for subagent failure?

- ☐ If a subagent fails, the entire workflow always fails
- ☒ Design for failure: use timeouts, handle partial results, and have fallback strategies

Correct! Correct! Subagents can timeout, produce errors, or return unexpected results. Robust workflows handle these cases.

- ☐ Assume all subagents will succeed; don't add error handling since it's just overhead
- ☐ If a subagent fails, immediately spawn 5 more to retry the same task

You got all the answers!

Question 4: How do Claude Code and Codex expose subagents?

- ☐ Claude Code: use `/agent spawn`. Codex: use `codex-agent spawn`. They're identical.
- ☒ Claude Code: conversational invocation + custom agents in `.claude/agents/`. Codex: natural language requests + custom agents as TOML files in `.codex/agents/`.

Correct! Correct! Both platforms use natural language to trigger subagent creation, but configure custom agents differently — Claude Code with markdown files, Codex with TOML files.

- ☐ Claude Code doesn't support subagents. Only Codex does.
- ☐ Both platforms use the exact same subagent API and configuration format

You got all the answers!

Question 5: What built-in agent types does Codex provide?

- ☒ Codex has three built-in agent types: default (general), worker (implementation), and explorer (read-heavy codebase exploration)



Correct! Correct! These three built-in types cover the most common subagent use cases without requiring custom agent definitions.

- ☒ Codex has no built-in agent types; you must always define custom agents
- ☒ Codex only supports one agent at a time; there's no parallelism
- ☒ Codex subagents require a separate codex-agent binary to manage

Submit

You got all the answers!

Summary

If you got 4-5 correct, you're ready to design multi-agent workflows without reaching for subagents unnecessarily. If you missed several, review the platform-specific invocation patterns and failure-handling guidance.

Key Takeaways

- Use subagents when the task shape justifies isolation or parallelism
- Pass only the context a child agent needs, and use files when the payload is too large for a prompt
- Design for retries, partial results, and platform-specific invocation details from the start

Next Steps

You've finished Unit 4: Subagents. Next up is Unit 5, where you'll add deterministic lifecycle hooks around those same agent workflows.

</> [Update on GitHub](#)

 Ask HuggingChat



← Hands-On: Multi-Agent Workflow

Unit 5: Hooks →



Ask HuggingChat

